Python E-Course

Module 4 — Color Spaces

Detailed Notes

Module Overview

Level: Color & Geometry

Duration: ~1 week

Prerequisites: Modules 1-3

What You'll Learn

- Convert between RGB/BGR, HSV, LAB, YCrCb.
- Read HSV histograms and pick color ranges for object detection.
- Choose the right color space for the task (segmentation, similarity, printing).
- Avoid the OpenCV HSV-range trap (H is 0–179, not 0–360).

Lessons

#	Lesson	Duration
1	Why Color Spaces Matter	25 min
2	RGB and BGR	20 min
3	HSV — the Object-Detection Workhorse	35 min
4	LAB and YCrCb	25 min
5	Converting with cv2.cvtColor	20 min
6	Color Picking for Object Detection	30 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 5: Geometric Transformations

Lesson 1: Why Color Spaces Matter

Module: 4 — Color Spaces

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Explain the limitations of RGB for perceptual tasks.
- Name the three main alternative color spaces and what each is good for.
- Predict which space to choose for: object detection by color, perceptual similarity, broadcast video.

Prerequisites

- Module 3

1. Why This Matters

RGB stores how much red, green, and blue light a pixel emits — great for displays, bad for "find every red apple". For most perceptual tasks (color matching, segmentation, lighting-invariant detection), an alternative color space makes the problem 10× easier.

2. Concept

2.1 RGB's weakness: lighting changes everything

A red apple in shade is (80, 30, 30). In sun it's (240, 90, 90). Both are red, but RGB values differ by 3×. Threshold-based "find red" in RGB has to deal with this constantly.

2.2 HSV separates color from lightness

HSV = Hue, Saturation, Value.

- H — what color is it? (0 = red, 60 = yellow, 120 = green, 240 = blue)
- S — how saturated / pure is it? (0 = gray, 255 = vivid)
- V — how bright is it? (0 = black, 255 = max)

Shade and sun change V (and a bit of S), but H stays the same. Filter on H → reliable color detection.

2.3 LAB is perceptually uniform

LAB: Lightness + a (green↔red) + b (blue↔yellow). Two LAB colors with the same Euclidean distance look about equally different to humans — RGB doesn't have this property. Use LAB when comparing colors meaningfully.

2.4 YCrCb separates luma from chroma

Y = brightness; Cr, Cb = color differences. This is how broadcast video (JPEG, MPEG) is stored — you can compress chroma more than luma without humans noticing. Useful for skin-tone detection too.

2.5 Cheat sheet

Goal	Use
Display	RGB / BGR
Color-based segmentation	HSV
Perceptual color distance	LAB
Broadcast / skin detection	YCrCb
Print	CMYK (not built into OpenCV)

3. Code Examples

Example 1: Same apple under two lights — RGB vs HSV

```
import numpy as np, cv2

# Synthesize: a 50x50 patch in two "lighting" conditions
shade = np.full((50, 50, 3), (30, 30, 80), dtype=np.uint8)    # BGR: dark red
sun   = np.full((50, 50, 3), (90, 90, 240), dtype=np.uint8)   # BGR: bright red

for name, img in [("shade", shade), ("sun", sun)]:
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    print(f"{name:5s}  BGR={tuple(img[0,0])}  HSV={tuple(hsv[0,0])}")
```

Expected Output:

```
shade  BGR=(30, 30, 80)   HSV=(0, 159, 80)
sun    BGR=(90, 90, 240)  HSV=(0, 159, 240)
```

Notice: H is the same (0 = red) in both. Only V (brightness) differs. That's why HSV is the choice for object detection by color.

Example 2: Show all three spaces side by side

```
import cv2, matplotlib.pyplot as plt
from skimage.data import coffee

bgr = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax[0].imshow(cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)); ax[0].set_title("RGB")
ax[1].imshow(hsv);  ax[1].set_title("HSV (raw channels as RGB)")
ax[2].imshow(lab);  ax[2].set_title("LAB (raw channels as RGB)")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

Expected Output: Three panels. The first is the normal coffee image. The other two are nonsensical pseudo-colored versions because we're displaying HSV/LAB values as if they were RGB — that's the point: these spaces aren't meant for direct display.

4. Parameter Sensitivity

(No params here — each conversion is deterministic.)

5. Common Mistakes

Mistake	What Happens	Fix
Filtering by RGB for object color	Fails under lighting changes	Convert to HSV first
Displaying HSV image with imshow	Looks psychedelic	Convert back to BGR/RGB for display
Assuming H is 0..360	OpenCV uses 0..179 (next lesson)	Halve textbook H values

6. Practice Tasks

- Predict HSV of pure red (0, 0, 255) BGR. Verify with cv2.cvtColor.
- Predict HSV of pure green (0, 255, 0) BGR. Verify.
- Convert astronaut() to LAB and print the L channel's mean.

7. Key Takeaways

- RGB is for display.
- HSV separates hue from brightness — best for color detection.
- LAB is perceptually uniform — best for color similarity.
- YCrCb separates luma from chroma — used in video / skin detection.

8. What's Next

The boring necessary one — RGB and BGR (and why every library disagrees).

Lesson 2: RGB and BGR

Module: 4 — Color Spaces

Duration: 20 minutes

Difficulty: Beginner

Learning Objectives

- Tell RGB from BGR in code.
- Convert between them with cv2.cvtColor or slicing.
- Recognise visual symptoms of using the wrong order.

Prerequisites

- Module 4 Lesson 1

1. Why This Matters

This is the single most common bug in beginner OpenCV code: load with cv2.imread (BGR), pass directly to matplotlib (expects RGB), and wonder why reds look blue. Settling this is a 20-minute lesson that saves a career of confusion.

2. Concept

2.1 Three channels, two orders

- RGB: index 0 = Red, 1 = Green, 2 = Blue. Standard everywhere (web, scikit-image, matplotlib, Pillow).
- BGR: index 0 = Blue, 1 = Green, 2 = Red. OpenCV's historical choice.

The image data is the same; the interpretation of channel order differs.

2.2 Library cheat sheet

Library	Order
OpenCV (cv2.imread, cv2.imwrite)	BGR
skimage (skimage.io.imread, skimage.data.*)	RGB
Pillow	RGB
matplotlib imshow	RGB
HTML / CSS	RGB

2.3 Conversions

```
rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
bgr = cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR)
```

Both are equivalent to swapping the last axis:

```
rgb = bgr[..., ::-1]    # reverses last axis
```

The slicing form is a view, the cvtColor form is a copy. For long-lived results prefer cvtColor.

2.4 Visual symptom

When you display BGR data as RGB:

- Reds appear blue
- Blues appear red
- Greens unchanged

Skin tones look ghastly; skies look orange.

3. Code Examples

Example 1: Same image, three orderings

```
import cv2, matplotlib.pyplot as plt
from skimage.data import astronaut

img_rgb = astronaut()                          # RGB
img_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax[0].imshow(img_rgb);
ax[0].set_title("RGB (correct)")
ax[1].imshow(img_bgr);
ax[1].set_title("BGR shown as RGB (WRONG)")
ax[2].imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB));
ax[2].set_title("BGR → RGB (correct)")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

Expected Output: Three panels. Panel 1 and 3 look identical (correct astronaut). Panel 2 has all the reds shifted to cyan / blue.

Example 2: Slicing vs cvtColor

```
import cv2, numpy as np
from skimage.data import coffee

bgr = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
rgb_a = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
rgb_b = bgr[..., ::-1]                             # view

print("equal:", np.array_equal(rgb_a, rgb_b))      # True
print("same memory:", rgb_a.base is None,
      "vs slice base shared?", rgb_b.base is bgr)
```

Expected Output:

```
equal: True
same memory: True vs slice base shared? True
```

Example 3: Define a tiny show helper

```
import cv2, matplotlib.pyplot as plt
```

```
def show_bgr(img, title=""):
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title(title); plt.axis("off"); plt.show()

# always safe — never display BGR directly
```

4. Parameter Sensitivity

(No params — choice is RGB vs BGR.)

5. Common Mistakes

Mistake	What Happens	Fix
plt.imshow(cv2.imread(...))	Reds → blues	Wrap with cv2.cvtColor(..., COLOR_BGR2RGB)
cv2.imwrite("x.jpg", rgb_image) from skimage	Reds and blues swapped in saved file	Convert RGB → BGR before saving
Mixing skimage and cv2 without converting	Channels swap silently	Convert at every boundary

6. Practice Tasks

- Load astronaut.png with cv2.imread and display with matplotlib without converting. Note how it looks.
- Convert and display correctly. Confirm helmet looks orange / red, not blue.
- Build a show(img) helper that auto-converts BGR before display.

7. Key Takeaways

- OpenCV = BGR. Everything else = RGB.
- Convert at library boundaries.
- cv2.cvtColor (copy) or [..., ::-1] (view).

8. What's Next

HSV — the color space you'll use most for object detection.

Lesson 3: HSV — the Object-Detection Workhorse

Module: 4 — Color Spaces

Duration: 35 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Convert BGR → HSV and back.
- Understand OpenCV's 0–179 H range (the trap).
- Build a binary mask with cv2.inRange.
- Pick HSV ranges for common colors.

Prerequisites

- Module 4 Lesson 2

1. Why This Matters

HSV is the color space for color-based object detection. Splitting hue from brightness gives you robustness to lighting changes that's impossible in RGB. The catch — OpenCV's H range is unusual.

2. Concept

2.1 The OpenCV ranges

Channel	Range	Meaning
H	0–179	Hue (red, yellow, green, …)
S	0–255	Saturation
V	0–255	Value (brightness)

The trap: Most references say H goes 0–360. OpenCV halves it to fit in uint8. So:

- Textbook H = 30 (orange) → OpenCV H = 15
- Textbook H = 120 (green) → OpenCV H = 60
- Textbook H = 240 (blue) → OpenCV H = 120

Halve any reference value before using it in cv2.inRange.

2.2 Color → H mapping

Color	OpenCV H
Red	0 or 179 (wraps around)
Orange	~15
Yellow	~30
Green	~60
Cyan	~90
Blue	~120

Magenta	~150
Red (other end)	179

Red is special — it spans both ends. You usually need two masks for red and OR them.

2.3 cv2.inRange for masking

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (low_h, low_s, low_v), (high_h, high_s, high_v))
```

Returns a uint8 mask: 255 where in range, 0 otherwise.

2.4 Saturation and Value cutoffs

Almost always include lower bounds on S and V to exclude grays and shadows. A typical color range:

```
# Yellow
low  = ( 20, 100, 100)
high = ( 40, 255, 255)
```

The 100, 100 lower bounds say "must be reasonably saturated and bright" — excludes washed-out grays.

2.5 The red wraparound

```
mask1 = cv2.inRange(hsv, (0,   100, 100), (10,  255, 255))
mask2 = cv2.inRange(hsv, (170, 100, 100), (179, 255, 255))
mask_red = cv2.bitwise_or(mask1, mask2)
```

3. Code Examples

Example 1: Pure colors → HSV values

```
import cv2, numpy as np

colors_bgr = {
    "Red":    (  0,   0, 255),
    "Green":  (  0, 255,   0),
    "Blue":   (255,   0,   0),
    "Yellow": (  0, 255, 255),
    "Cyan":   (255, 255,   0),
}

for name, bgr in colors_bgr.items():
    px = np.full((1, 1, 3), bgr, dtype=np.uint8)
    h, s, v = cv2.cvtColor(px, cv2.COLOR_BGR2HSV)[0, 0]
    print(f"{name:7s} BGR={bgr}  ->  H={h:3d}  S={s:3d}  V={v:3d}")
```

Expected Output:

```
Red     BGR=(0, 0, 255)    ->  H=  0  S=255  V=255
Green   BGR=(0, 255, 0)    ->  H= 60  S=255  V=255
Blue    BGR=(255, 0, 0)    ->  H=120  S=255  V=255
Yellow  BGR=(0, 255, 255)  ->  H= 30  S=255  V=255
Cyan    BGR=(255, 255, 0)  ->  H= 90  S=255  V=255
```

Example 2: Mask green pixels with inRange

```
import cv2, numpy as np

# build a synthetic image: half green, half blue
img = np.zeros((200, 400, 3), dtype=np.uint8)
img[:, :200] = (0, 255, 0)
img[:, 200:] = (255, 0, 0)

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (40, 100, 100), (80, 255, 255))    # green range

print("green pixels:", (mask == 255).sum(), "out of", mask.size)
cv2.imwrite("mask_green.png", mask)
```

Expected Output: green pixels: 40000 out of 80000 and a mask_green.png that's white on the left half, black on the right.

Example 3: Detect red — with wraparound

```
import cv2, numpy as np

img = cv2.cvtColor(np.tile(np.array(
    [[(0, 0, 255), (0, 0, 200), (0, 0, 150)]], dtype=np.uint8), (50, 1, 1)),
cv2.COLOR_RGB2BGR)
# (just a synthetic red stripe)

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
m1 = cv2.inRange(hsv, (0,   100, 50), (10,  255, 255))
m2 = cv2.inRange(hsv, (170, 100, 50), (179, 255, 255))
mask = cv2.bitwise_or(m1, m2)
print("red pixels detected:", (mask == 255).sum())
```

Expected Output: A non-zero count showing that the red region was captured by combining the two ranges.

Example 4: Real image — find red objects in a photo

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

m1 = cv2.inRange(hsv, (0,   90, 50), (10,  255, 255))
m2 = cv2.inRange(hsv, (170, 90, 50), (179, 255, 255))
mask = cv2.bitwise_or(m1, m2)

result = cv2.bitwise_and(img, img, mask=mask)
cv2.imwrite("coffee_red_only.png", result)
print("red pixel fraction:", (mask > 0).mean())
```

Expected Output: coffee_red_only.png shows only the red parts of the coffee image (cherry / berry tones in the bean shadows). The rest is black. Console prints something like red pixel fraction: 0.08.

4. Parameter Sensitivity

Param	Lower	Higher
H low/high	tighter range (specific color)	broader range (more shades)
S low	includes pale / washed colors (more noise)	only vivid colors
V low	includes shadows	only bright objects

5. Common Mistakes

Mistake	What Happens	Fix
Using textbook H (0–360) directly	Wrong color matched	Halve to OpenCV 0–179
Red without wraparound	Misses half the reds	Two masks + OR
Too-low S/V bounds	Captures gray pixels	Set S, V >= 70–100
Forgetting to convert BGR→HSV	Garbage matches	cv2.cvtColor(img, cv2.COLOR_BGR2HSV) first

6. Practice Tasks

- Compute HSV for pure orange BGR (0, 128, 255). What H?
- Build a mask for "blue" pixels in astronaut() (the suit). Tune ranges visually.
- Build a red mask with wraparound; apply to coffee().

7. Key Takeaways

- HSV separates color (H) from brightness (V).
- OpenCV H is 0–179. Halve textbook values.
- cv2.inRange + tight S/V bounds = the classic color filter.
- Red wraps; use two ranges + OR.

8. What's Next

LAB and YCrCb — when perceptual distance or chroma separation matters.

Lesson 4: LAB and YCrCb

Module: 4 — Color Spaces

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Convert to/from LAB and YCrCb.
- Use LAB for perceptual color distance and lightness manipulation.
- Use YCrCb for skin detection and chroma-separated processing.
- Know which channel of each space carries which information.

Prerequisites

- Module 4 Lesson 3

1. Why This Matters

HSV is good but not perceptually uniform. LAB is, which makes it the right space for "are these two colors similar to a human?". YCrCb separates brightness from chroma — handy when you want to manipulate one without the other (the basis for JPEG compression and many skin-detection pipelines).

2. Concept

2.1 LAB

Channel	Range (OpenCV uint8)	Meaning
L	0–255	Lightness (0 = black, 255 = white)
a	0–255	green ↔ red (128 = neutral)
b	0–255	blue ↔ yellow (128 = neutral)

(In the academic LAB definition, L is 0–100 and a/b are signed. OpenCV scales them to fit uint8.)

Two LAB colors that differ by the same Euclidean distance look about equally different to the human eye. Use this for color similarity / nearest-color matching.

2.2 Common LAB recipes

CLAHE on L channel only (improves contrast without color shift — covered fully in Module 6):

```
lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
L, a, b = cv2.split(lab)
L_eq = cv2.createCLAHE(clipLimit=2.0).apply(L)
lab_eq = cv2.merge([L_eq, a, b])
out = cv2.cvtColor(lab_eq, cv2.COLOR_LAB2BGR)
```

2.3 YCrCb

Channel	Meaning
Y	Luma (grayscale-like)
Cr	Red-difference chroma
Cb	Blue-difference chroma

YCrCb separates brightness from color. Used in JPEG/MPEG (which can compress chroma harder than luma) and in skin detection (skin tends to fall in a tight Cr/Cb range across many ethnicities).

Skin range (rough): Y ∈ [0, 255], Cr ∈ [133, 173], Cb ∈ [77, 127].

2.4 Pick the space for the task

Goal	Space
Make image more contrasty without shifting hue	LAB (CLAHE on L)
Find nearest color from a palette	LAB (Euclidean distance)
Detect skin in arbitrary lighting	YCrCb
Detect a specific color object	HSV
Convert for display	RGB / BGR

3. Code Examples

Example 1: Convert and inspect ranges

```
import cv2, numpy as np
from skimage.data import astronaut

bgr = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)
ycc = cv2.cvtColor(bgr, cv2.COLOR_BGR2YCrCb)

for name, img in [("LAB", lab), ("YCrCb", ycc)]:
    for i, ch in enumerate(["c1", "c2", "c3"]):
        v = img[..., i]
        print(f"{name} {ch}: min={v.min()} max={v.max()} mean={v.mean():.1f}")
    print()
```

Expected Output (rough values, depending on the image):

```
LAB c1: min=2 max=255 mean=124.2
LAB c2: min=88 max=183 mean=132.1
LAB c3: min=86 max=192 mean=148.7

YCrCb c1: min=14 max=251 mean=105.4
YCrCb c2: min=104 max=181 mean=137.1
YCrCb c3: min=90 max=151 mean=123.5
```

Example 2: Boost contrast via LAB (no hue shift)

```
import cv2
from skimage.data import coffee
```

```
bgr = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)
L, a, b = cv2.split(lab)
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
L_eq = clahe.apply(L)
lab_eq = cv2.merge([L_eq, a, b])
out = cv2.cvtColor(lab_eq, cv2.COLOR_LAB2BGR)
cv2.imwrite("coffee_clahe_lab.png", out)
```

Expected Output: Coffee photo with sharper contrast in highlights / shadows, but colors are not over-saturated (unlike doing CLAHE per RGB channel, which can produce odd color shifts).

Example 3: Skin detection in YCrCb

```
import cv2, numpy as np
from skimage.data import astronaut

bgr = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
ycc = cv2.cvtColor(bgr, cv2.COLOR_BGR2YCrCb)

# Loose skin range
skin = cv2.inRange(ycc, (0, 133, 77), (255, 173, 127))
out = cv2.bitwise_and(bgr, bgr, mask=skin)
cv2.imwrite("astro_skin.png", out)
print("skin pixel fraction:", (skin > 0).mean())
```

Expected Output: astro_skin.png shows mostly the astronaut's face (and any other skin-tone regions) — the rest is black. Console prints something like skin pixel fraction: 0.04.

Example 4: Color-similarity in LAB

```
import cv2, numpy as np

# Palette and target color (BGR)
palette_bgr = [(0,0,255), (255,0,0), (0,255,0), (200,200,200)]
target_bgr  = (40, 30, 220)   # a slightly off-red

palette_lab = [cv2.cvtColor(np.array([[c]], dtype=np.uint8), cv2.COLOR_BGR2LAB)[0,0]
for c in palette_bgr]
target_lab  = cv2.cvtColor(np.array([[target_bgr]], dtype=np.uint8),
cv2.COLOR_BGR2LAB)[0,0].astype(np.int32)

distances = [np.linalg.norm(target_lab - p.astype(np.int32)) for p in palette_lab]
best = int(np.argmin(distances))
print("nearest in palette:", palette_bgr[best], " distance:", distances[best])
```

Expected Output: Tells you the off-red target is closest to the pure-red (0,0,255) palette entry — and the distance number quantifies how close.

4. Parameter Sensitivity

(Conversions are fixed; no params beyond which channel you operate on.)

5. Common Mistakes

Mistake	What Happens	Fix
Forgetting LAB's a/b are centred at 128	Sign confusion	Subtract 128 if you need signed values
Doing CLAHE on RGB channels independently	Color shift	Do CLAHE on L of LAB, leave a/b alone
Using LAB for arbitrary "color detection"	Less convenient than HSV	HSV for detection; LAB for similarity

6. Practice Tasks

- Convert coffee() to LAB; print mean of L channel.
- Build a YCrCb skin mask on astronaut(). Inspect coverage.
- Find the nearest of [red, green, blue] to an off-orange BGR color, using LAB distance.

7. Key Takeaways

- LAB: perceptually uniform; use for color similarity and contrast manipulation.
- YCrCb: luma + chroma; useful for skin detection and video.
- Different tools for different goals — pick based on the question you're answering.

8. What's Next

The conversion API itself — cv2.cvtColor and the constants you'll meet.

Lesson 5: Converting with cv2.cvtColor

Module: 4 — Color Spaces

Duration: 20 minutes

Difficulty: Beginner

Learning Objectives

- Use cv2.cvtColor for any conversion you need.
- Remember the most-used conversion flags.
- Convert grayscale ↔ color, do alpha extraction.

Prerequisites

- Module 4 Lesson 4

1. Why This Matters

cv2.cvtColor is the single function for almost every color-space change. Memorising 6 conversion flags covers 95% of real-world work.

2. Concept

2.1 Signature

```
out = cv2.cvtColor(src, code)
```

Where code is a constant like cv2.COLOR_BGR2GRAY. The result has whatever shape the destination space requires (e.g. (H, W) for grayscale, (H, W, 3) for color).

2.2 The flags you'll actually use

Conversion	Flag
BGR → RGB	COLOR_BGR2RGB
RGB → BGR	COLOR_RGB2BGR
BGR → grayscale	COLOR_BGR2GRAY
RGB → grayscale	COLOR_RGB2GRAY
gray → BGR (replicate)	COLOR_GRAY2BGR
BGR → HSV	COLOR_BGR2HSV
HSV → BGR	COLOR_HSV2BGR
BGR → LAB	COLOR_BGR2LAB
LAB → BGR	COLOR_LAB2BGR
BGR → YCrCb	COLOR_BGR2YCrCb
YCrCb → BGR	COLOR_YCrCb2BGR
BGRA → BGR	COLOR_BGRA2BGR (drop alpha)
BGRA → alpha only	(slice: bgra[..., 3])

2.3 Grayscale conversion math

COLOR_BGR2GRAY uses the ITU-R BT.601 weights:

```
gray = 0.299 R + 0.587 G + 0.114 B
```

The exact pure-NumPy form (covered in M1 L5):

```
gray = (img.astype(np.float32) * (0.114, 0.587, 0.299)).sum(axis=2).clip(0,
255).astype(np.uint8)
```

(Weights in BGR order to match cv2.cvtColor's input.)

2.4 Gray → BGR

```
bgr = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
```

This just replicates the gray value into 3 channels — useful when you need to draw a colored annotation on a grayscale background.

2.5 Multi-step pipelines

```
img = cv2.imread("in.jpg")                    # BGR
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, lo, hi)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray_masked = cv2.bitwise_and(gray, gray, mask=mask)
```

3. Code Examples

Example 1: BGR ↔ grayscale round-trip

```
import cv2, numpy as np
from skimage.data import astronaut

bgr = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
bgr2 = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)

print("bgr:", bgr.shape, "gray:", gray.shape, "back:", bgr2.shape)
# Channels of bgr2 are replicated, not the original — round-trip is lossy
print("identical?", np.array_equal(bgr, bgr2))
```

Expected Output:

```
bgr: (512, 512, 3) gray: (512, 512) back: (512, 512, 3)
identical? False
```

Example 2: Quick HSV pipeline

```
import cv2, numpy as np
from skimage.data import coffee

bgr = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (0, 0, 0), (179, 255, 100))   # all dark pixels
print("dark pixel fraction:", (mask > 0).mean())
cv2.imwrite("coffee_dark_mask.png", mask)
```

Expected Output: dark pixel fraction: ~0.20 (depending on the image) and a mask highlighting the darkest regions.

Example 3: Alpha extraction

```
import cv2, numpy as np

# Build BGRA image
bgra = np.zeros((100, 100, 4), dtype=np.uint8)
bgra[:, :, 2] = 255           # red
bgra[:, :, 3] = np.tile(np.linspace(0, 255, 100, dtype=np.uint8), (100, 1))

bgr = cv2.cvtColor(bgra, cv2.COLOR_BGRA2BGR)
alpha = bgra[..., 3]
print("bgr shape:", bgr.shape, "alpha shape:", alpha.shape)
```

Expected Output:

```
bgr shape: (100, 100, 3) alpha shape: (100, 100)
```

4. Parameter Sensitivity

(Each code is a fixed conversion — no params.)

5. Common Mistakes

Mistake	What Happens	Fix
Wrong code direction	Wrong color or shape	Read direction in the flag name: BGR2GRAY not GRAY2BGR
Converting an already-gray image to gray	OpenCV may error or no-op	Check img.ndim first
Forgetting the result shape changes	Crash on later code expecting (H, W, 3)	Branch on shape after conversion

6. Practice Tasks

- Convert coffee() to grayscale and back to BGR; compare round-trip.
- Build a quick HSV → mask → result pipeline in 3 lines.
- Extract alpha from a PNG-with-alpha and save it as a grayscale image.

7. Key Takeaways

- cv2.cvtColor(src, code) covers nearly all color conversions.
- Remember code direction (SRC2DST).
- BGR2GRAY is the standard grayscale conversion.
- GRAY2BGR just replicates — round-trip is lossy.

8. What's Next

Color picking — turning an HSV understanding into actual object detection.

Lesson 6: Color Picking for Object Detection

Module: 4 — Color Spaces

Duration: 30 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Pick a target color from an image interactively.
- Choose HSV ranges that capture an object across lighting variation.
- Build an end-to-end detect-and-highlight pipeline.

Prerequisites

- Module 4 Lesson 5

1. Why This Matters

Hand-tuning HSV ranges is the most common debug step in color-based detection. Knowing a repeatable picker workflow saves hours.

2. Concept

2.1 The picker workflow

- Sample: take a small patch of pixels from a representative object.
- Convert to HSV.
- Measure: compute min/max/mean of each channel across the patch.
- Pad: widen the range a bit to handle variation.
- Apply: cv2.inRange over the whole image.
- Inspect: are too many or too few pixels selected? Iterate.

2.2 Sampling helper

```
def sample_hsv(img_bgr, x, y, half=5):
    patch = img_bgr[y - half:y + half, x - half:x + half]
    hsv = cv2.cvtColor(patch, cv2.COLOR_BGR2HSV).reshape(-1, 3)
    h, s, v = hsv[:, 0], hsv[:, 1], hsv[:, 2]
    return {"h": (int(h.min()), int(h.max())),
            "s": (int(s.min()), int(s.max())),
            "v": (int(v.min()), int(v.max())),
            "mean": (int(h.mean()), int(s.mean()), int(v.mean()))}
```

2.3 Range padding heuristic

For a sampled mean and spread:

```
H ± 10
S ∈ [max(50, S_min - 30), 255]
V ∈ [max(50, V_min - 30), 255]
```

These are starting values — tune per image.

2.4 Interactive picker with mouse callback

```
import cv2

img = cv2.imread("photo.jpg")
def on_mouse(event, x, y, flags, param):
    if event == cv2.EVENT_LBUTTONDOWN:
        sample = sample_hsv(img, x, y)
        print(sample)

cv2.namedWindow("pick")
cv2.setMouseCallback("pick", on_mouse)
while True:
    cv2.imshow("pick", img)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cv2.destroyAllWindows()
```

Click on the object to get its HSV ranges; ctrl-C to exit.

2.5 Mask post-processing

After cv2.inRange, masks often have small holes / specks. Clean with morphology (preview of Module 10):

```
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN,  kernel)   # remove specks
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)   # fill holes
```

2.6 Highlighting detection

Common patterns:

- cv2.bitwise_and(img, img, mask=mask) — show only the object.
- cv2.bitwise_not(mask) and apply to background — desaturate / grey-out everything else.
- Draw bounding box around the largest contour of the mask (Module 12).

3. Code Examples

Example 1: Detect red apples — full pipeline

```
import cv2, numpy as np
from skimage.data import coffee   # stand-in: red bean tones

bgr = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)

# red ranges
m1 = cv2.inRange(hsv, (0,   80, 50), (10,  255, 255))
m2 = cv2.inRange(hsv, (170, 80, 50), (179, 255, 255))
mask = cv2.bitwise_or(m1, m2)

# clean
```

```
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

# highlight: keep red, darken the rest
dark_bg = (bgr * 0.3).astype(np.uint8)
fg = cv2.bitwise_and(bgr, bgr, mask=mask)
bg = cv2.bitwise_and(dark_bg, dark_bg, mask=cv2.bitwise_not(mask))
result = cv2.add(fg, bg)
cv2.imwrite("coffee_red_highlight.png", result)
print("red pixel fraction:", (mask > 0).mean())
```

Expected Output: coffee_red_highlight.png — the original image but with anything not-red darkened to 30% brightness; reds pop out.

Example 2: HSV sampler function in action

```
import cv2, numpy as np
from skimage.data import astronaut

bgr = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)

def sample_hsv(img_bgr, x, y, half=5):
    patch = img_bgr[y - half:y + half, x - half:x + half]
    hsv = cv2.cvtColor(patch, cv2.COLOR_BGR2HSV).reshape(-1, 3)
    h, s, v = hsv[:, 0], hsv[:, 1], hsv[:, 2]
    return dict(h_min=int(h.min()), h_max=int(h.max()),
                s_min=int(s.min()), s_max=int(s.max()),
                v_min=int(v.min()), v_max=int(v.max()))

# Sample a known patch (somewhere on the orange helmet area)
sample = sample_hsv(bgr, x=250, y=180)
print("sample:", sample)
```

Expected Output (something like):

```
sample: {'h_min': 5, 'h_max': 22, 's_min': 120, 's_max': 210, 'v_min': 100, 'v_max':
200}
```

Use these as the basis for a tuned cv2.inRange call.

Example 3: Mask cleanup demo

```
import cv2, numpy as np

# Noisy mask
np.random.seed(0)
mask = (np.random.rand(200, 200) < 0.2).astype(np.uint8) * 255
cv2.imwrite("mask_raw.png", mask)

k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
cleaned = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k)
cv2.imwrite("mask_opened.png", cleaned)

print("raw white pixels:    ", (mask > 0).sum())
print("opened white pixels: ", (cleaned > 0).sum())
```

Expected Output: Far fewer "white" pixels after opening — small specks removed.

4. Parameter Sensitivity

Adjustment	Effect
Widen H by ±5	Catch more shades
Lower S min	Include washed colors (and grays!)
Lower V min	Include shadowed object parts (and darkness)
Bigger morph kernel	Remove more specks but eat into the object

5. Common Mistakes

Mistake	What Happens	Fix
Tuning H way too tight	Misses the object under different lighting	Sample multiple patches; pad
S, V min = 0	Lots of false positives	Set ≥ 50–100
Skipping morphology	Mask is noisy, lots of specks	Open then close
Sampling a single pixel	Bad statistics	Sample a 10×10 patch min, mean, max

6. Practice Tasks

- Take any image with a clearly red object. Sample its center; print HSV stats.
- Tune an inRange to detect that object; show the masked-out highlight.
- Apply MORPH_OPEN then MORPH_CLOSE and compare the mask before/after.

7. Key Takeaways

- Sample a patch (not a pixel) to get range stats.
- Pad H by ±10, set S/V mins ≥ 50–100.
- Always clean the mask with morphology.
- Iterate visually — color picking is an empirical loop.

8. What's Next

End of Module 4. Next: Module 5 — Geometric Transformations.

Module 4 — Exercises

18 exercises (3 per lesson).

4.1 Why

- (E) For pure red BGR (0,0,255), predict and verify HSV.
- (M) Predict that two patches of red (dark/bright) have the same H. Verify.
- (H) Identify which color space you'd use to: (a) detect a green ball, (b) compare two grays for similarity, (c) compress chroma. Justify each.

4.2 RGB/BGR

- (E) Load astronaut.png with cv2.imread; display with matplotlib without conversion.
- (M) Convert and display correctly.
- (H) Build a show(img) helper that auto-converts BGR before display.

4.3 HSV

- (E) Compute HSV for orange (0, 128, 255) BGR.
- (M) Build a blue mask on astronaut() (the suit).
- (H) Build a red mask with wraparound on coffee().

4.4 LAB / YCrCb

- (E) Convert coffee() to LAB; print mean of L channel.
- (M) Build a YCrCb skin mask on astronaut(); print coverage.
- (H) Find nearest of [red, green, blue] palette colors to off-orange via LAB distance.

4.5 cvtColor

- (E) Convert coffee() to gray and back to BGR. Are they identical? Why not.
- (M) Build a 3-step pipeline: BGR → HSV → inRange (dark) → save mask.
- (H) Extract the alpha channel from a PNG-with-alpha. Save as grayscale.

4.6 Color Picking

- (E) Sample a 10×10 HSV patch from any image. Print min/max/mean per channel.
- (M) Tune inRange to detect a red object in your own photo.
- (H) Apply MORPH_OPEN + MORPH_CLOSE to clean a noisy mask; compare counts.

Module 4 — Quiz

10 MCQs.

Q1

OpenCV's H channel range is: A. 0–360 B. 0–255 C. 0–179 D. 0–100

Answer: C (L3)

Q2

The S channel measures: A. brightness B. saturation C. hue D. shadow

Answer: B (L3)

Q3

For red detection in OpenCV HSV, you typically use: A. one range around H=0 B. one range around H=179 C. two ranges (around 0 and 179) OR'd D. H=120

Answer: C — red wraps. (L3)

Q4

LAB's L channel is: A. red↔green B. blue↔yellow C. lightness D. saturation

Answer: C (L4)

Q5

For perceptual color distance, use: A. RGB B. HSV C. LAB D. BGR

Answer: C (L4)

Q6

For skin detection, a good color space is: A. RGB B. YCrCb C. CMYK D. HSV palette only

Answer: B (L4)

Q7

cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) produces: A. (H, W, 3) B. (H, W) C. (H, W, 1) D. error

Answer: B (L5)

Q8

cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR): A. recovers original color B. replicates gray into 3 channels C. errors D. inverts

Answer: B (L5)

Q9

To detect a specific color robustly across lighting, the best space is: A. RGB B. HSV C. LAB D. YCrCb

Answer: B (L1, L3)

Q10

Why include lower S/V bounds in cv2.inRange? A. Speeds up B. Excludes grays / shadows C. Saves memory D. Required by API

Answer: B (L6)

Module 4 — Assignment: Color-Based Object Highlighter

Estimated time: 2 hours

Combines: Module 4 + Modules 2-3

Brief

CLI tool that takes an image and a target color name (red/green/blue/yellow/orange), detects all matching regions in HSV, and produces an output where the rest of the image is desaturated.

Requirements

- highlight.py INPUT --color red|green|blue|yellow|orange [--output OUT] [--desaturate 0.3]
- Hardcoded HSV ranges per color (your tuned values).
- Build the mask, clean with morphology open + close.
- Compose output:
- Where mask is 255: original pixels.
- Where mask is 0: original × --desaturate (default 0.3 = 30% brightness).
- Print: pixel coverage (% of image mask covers).

Constraints

- Module 4 + previous concepts only.
- Use named constants for color HSV ranges (dict).
- Handle the red wraparound.

Expected Behaviour (sample)

```
$ python highlight.py coffee.png --color red
saved -> coffee_highlight.png
red coverage: 8.2%
```

The output PNG shows the red regions of the coffee bean at full brightness; everything else dimmed.

Grading Rubric

Criterion	Weight
All 5 colors mapped	25%
Red wraparound handled	15%
Morphology cleanup	15%
Composite correct	25%
Style + argparse	20%

Submission

Save as assignment_module4.py.

Module 4 — Mini Project: HSV Color Picker (Trackbars)

Overview

Interactive cv2-based tool: load an image and adjust H/S/V min/max via trackbars. The mask and the masked-output update in real time. Once tuned, the tool prints the chosen range to stdout for re-use.

Concepts Used

- BGR → HSV (M4 L3)
- cv2.inRange masking (M3 L6 + M4 L3)
- cv2.imshow + trackbars (M2 L3)

Features

- 6 trackbars: H_low, H_high, S_low, S_high, V_low, V_high.
- Real-time mask preview.
- Real-time "masked image" preview.
- Press p → print current range to console.
- Press q → quit.

Starter Code

hsv_picker.py:

```
import sys
import cv2
import numpy as np

def nothing(_):
    pass

def main(path):
    img = cv2.imread(path)
    if img is None:
        raise FileNotFoundError(path)
    img = cv2.resize(img, None, fx=0.5, fy=0.5) if max(img.shape) > 800 else img
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

    cv2.namedWindow("controls")
    for name, init in [("H_low", 0), ("H_high", 179),
                       ("S_low", 0), ("S_high", 255),
                       ("V_low", 0), ("V_high", 255)]:
        cv2.createTrackbar(name, "controls",
                           init, 255 if "H" not in name else 179, nothing)

    while True:
        lo = (cv2.getTrackbarPos("H_low",  "controls"),
              cv2.getTrackbarPos("S_low",  "controls"),
              cv2.getTrackbarPos("V_low",  "controls"))
        hi = (cv2.getTrackbarPos("H_high", "controls"),
```

```
                cv2.getTrackbarPos("S_high", "controls"),
                cv2.getTrackbarPos("V_high", "controls"))

            mask = cv2.inRange(hsv, lo, hi)
            masked = cv2.bitwise_and(img, img, mask=mask)
            combo = np.hstack([img, cv2.cvtColor(mask, cv2.COLOR_GRAY2BGR), masked])
            cv2.imshow("controls", combo)

            key = cv2.waitKey(30) & 0xFF
            if key == ord('q'):
                break
            if key == ord('p'):
                print(f"low={lo}  high={hi}")

    cv2.destroyAllWindows()

if __name__ == "__main__":
    main(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/coffee.png")
```

Expected Interaction

```
$ python hsv_picker.py datasets/starter/coffee.png
(window opens; drag trackbars to find red regions; press p)
low=(0, 80, 50)  high=(10, 255, 255)
press q to quit
```

Extension Challenges

- Add a second set of trackbars and OR the two masks (helps with the red wraparound).
- Save the current mask and result when s pressed.
- Auto-load multiple images and let user cycle with arrow keys.

Grading Rubric

Criterion	Weight
Trackbars work	25%
Real-time preview	25%
Mask + masked side-by-side	20%
Print key (p)	15%
Code clarity	15%